

Algorithms 2

by Daniel Rosenberg and Michael Trushkin

These are lecture notes for the course Algorithms and Optimization at Ariel University. Students are assumed to have basic knowledge of graph theory, crucial definitions will be reminded. While we strive for accuracy, these notes may contain mistakes. Students are encouraged to report any errors or typos they encounter.

1. Polynomial-time algorithms

An algorithm is called *polynomial-time* if its running time is bounded by $O(n^c)$ where n is the length of the input and c is some (maybe huge) constant.

Example.

Common examples of polynomial-time algorithms include: DFS, BFS, Dijkstra's algorithm, 2-Coloring, and various sorting algorithms.

Definition 1.1.

$\mathbf{P} :=$ The set of problems that have a polynomial algorithm.

In this course we focus on computational complexity, and ignore the exact running time of the algorithms. Any polynomial-time algorithm is considered “efficient,” and we do not distinguish between different polynomial running times.

2. Self reduction

There are two types of problems: *decision problems* and *search problems*. Decision problems are those that require a ‘yes’ or ‘no’ answer, whereas search problems require finding an actual solution if one exists. For example, if the problem is finding a path between two given nodes A and B . The decision problem will be “**Is** there a path between node A and node B ?” and the search problem will be “**What** is the path between node A and B ?” both of these can be solved by the same algorithm. A non trivial¹ problem is finding k -clique, given a graph G , the decision problem is:

Questions 2.1.

K-CLIQUE: Is there a clique of size k in G ?

Questions 2.2.

SEARCH-k-CLIQUE: What is the clique of size k in G ?

¹None trivial in the sense that a greedy algorithm going over all vertices of size k results in algorithm with running time $\theta(n^k)$. Since, $k = \sqrt{n}$ is possible in that case we have running time of $\theta(n^{\sqrt{n}})$ which is exponential and not polynomial.

One might ask whether the problems are equivalent, by that we mean given a way to solve one, can we solve the other? By the nature of search problems if we have a way to find the solution, it is easy to answer whether there is a solution. The other way seems more complicated, but as it turns out the difference between both problems is polynomial.

Claim 2.1.

If the decision problem for k -clique can be solved in polynomial time, then there is a polynomial-time algorithm for SEARCH- k -CLIQUE.

Proof. In order to prove the claim, we will use the assumption that there is a polynomial-time algorithm for k -clique and show a polynomial time algorithm for SEARCH- k -CLIQUE. We propose the following algorithm:

Algorithm 1:

```

1: procedure SELF-REDUCTION( $G$ )
2:   if  $A(G) = 0$  then
3:     return null
4:   end
5:
6:   while  $v(G) > k$  do
7:     pick  $v \in V(G)$ 
8:     if  $A(G - v) = 1$  then
9:        $G \leftarrow G - v$ 
10:    end
11:  end
12:  return  $G$ 
13: end

```

If G does not have a valid clique of size k , then Algorithm 1 will return null on line 3 as required. Otherwise, we know that there is a clique of size k in G (maybe more than one), if at any point by removing some vertex $v \in V(G)$ we get that $A(G - v) = 0$ we know that v is essential to the clique and leave it in $V(G)$. Since at all times $A(G) = 1$, when $v(G) = k$ the algorithm stops and return G ; which is a clique of size k .

Moving to the complexity of Algorithm 1. By assumption A has complexity time bounded by some $f(G) = O(n^c)$ for some constant $c > 0$. The algorithm performs at most $n - k + 1$ calls to algorithm A where the first call at line 2 and at most $n - k$ calls at line 8. It therefore follows that the running time of Algorithm 1 is at most $(n - k + 1) \cdot f(G) = O(n^{c+1})$ which is polynomial. $\square$

The claim above demonstrates that decision and search problem are equivalent², thus we can focus only on decision problems.

²In polynomial time, which is what we mean by equivalent.

3. NP-completeness

While the class **P** contains a large portion of the problems students have faced so far, as it turns out the majority of the problems are not easy at all.

The story below captures the essence of our next complexity class **NP**, the set of problems where a proposed solution can be verified quickly:

Suppose you find an old antique safe and want to see what is inside. Breaking it open would probably damage whatever it contains, so the only option is to enter the correct code. But how do you know that such a code even exists? Perhaps the safe is broken, and no code will ever open it. On the other hand, if the safe is functioning as intended, then there must be some combination that unlocks it.

If someone were to hand you the correct code, you could easily check it—just type it in and see whether the safe opens. In that case, the code serves as a witness that the safe can indeed be opened. But if no such code exists, then there is no witness at all, and the safe is simply uncrackable.

Definition 3.1 (NP class).

A language³ L is said to be in **NP** if we have a polynomial-time algorithm M such that

$$x \in L \Leftrightarrow \exists y \text{ s.t. } |y| < p(|x|) \text{ and } M(x, y) = 1$$

where p is some polynomial

Remark 3.1.

In most literature y is called a *witness* and M is called *verifying algorithm*, where y plays the role of the answer, and M should just verify if the answer is correct.

In our story, y is the secret safe code, and you play the role of M verifying that y is indeed the correct code.

We are ready to meet our first **NP** language

Claim 3.1.

k -CLIQUE is in **NP**

Proof. To prove this, we are required to provide a polynomial-time verifier M that takes as an input a graph G and a witness y . Since the choice of y is up to us, we let $y = Y$ encode a set of vertices in G , and M verifies that Y is a clique of size k in G .

³Language is a set of words where words can be anything graphs, boolean assignments algorithms etc... In some courses language is defined as a subset of $\{0, 1\}^*$ i.e. the set of all binary strings of any length, this definition is identical, as anything can be encoded into a binary string.

Algorithm 2: Verifying algorithm for k -clique

```

1: procedure  $M(G, Y)$ 
2:    $\triangleright$  Check if the set of the correct size
3:   if  $|Y| \neq k$  then
4:     return false
5:   end
6:
7:    $\triangleright$  Check that all the vertices are real
8:   for  $v \in Y$  do
9:     if  $v \notin V(G)$  then
10:      return false
11:    end
12:  end
13:
14:    $\triangleright$  Check that all the edges exist
15:   for  $v, u \in Y$  do
16:     if  $(v, u) \notin E(G)$  then
17:       return false
18:     end
19:   end
20:
21:   return true
22: end

```

if $G \in k$ -clique, then there is a subset $V' \subseteq V(G)$ such that $G[V'] \cong K_k$, and $M(G, V') = 1$ if $G \notin k$ -clique, then no matter which subset $V' \subseteq V(G)$ we take, $G[V']$ will never be a clique in G . That is Y' either will have too many/little vertices, have “fake” vertices or there will be some missing edges, so that $M(G, V') = 0$.

The algorithm above clearly runs in $O(2k) + O(k^2)$ time which is bounded by $|V(G)|^2$ as $k \leq n$,⁴ which is polynomial. $\square$

4. Reductions

Suppose we have two languages/problems L_1, L_2 , can we know which one of them is *harder*? The intuition is that if L_2 harder than L_1 we would be able to solve L_2 using L_1 . This is done by “translating” our problem from L_1 to L_2 , solving the translated L_1 problem, and then answering accordingly. The translation between languages is called a *reduction*, formally

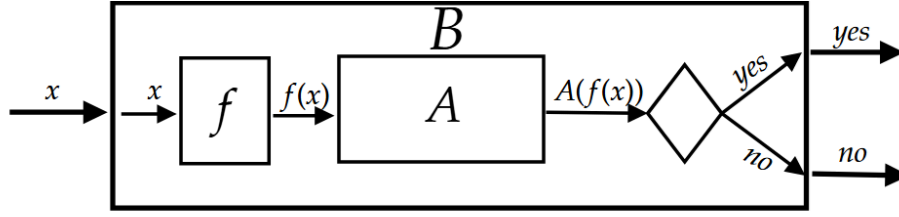
Definition 4.1 (polynomial time reduction).

Given two languages $L_1, L_2 \in \mathbf{NP}$, we write $L_1 \leq_p L_2$ if there exists a function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ and a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$, such that:

- $x \in L_1 \Leftrightarrow f(x) \in L_2$
- for every $x \in \{0, 1\}^*$, f runs in $p(|x|)$ time.

Assuming that $L_1 \leq_p L_2$ and given the polynomial reduction f as well as a black box A that solves L_2 . We can create an algorithm B that solves L_1 using only f and A in the following way:

⁴If $k > n$ then obviously no clique of size k exists.



Since the new algorithm only needs to call f , then pass $f(x)$ to A and answer similarly, we got an algorithm for B with similar time complexity as A . Now if A is polynomial, then B is also polynomial.

Definition 4.2 (NP-hard).

A language $L \subseteq \{0, 1\}^*$ is said to be NP-hard if $L' \leq_p L$ for every $L' \in \mathbf{NP}$

Intuitively, the following question arises:

Questions 4.1.

Are there any languages that are NP-Hard?

Example.

The language

$$L^* = \{(M', x, 1^c) : M'(x') = 1 \wedge M'(x') \text{ computes in } O(2^{|x|^c}) \text{ time}\}$$

is NP-hard.

Without going into too many details. Fix $L \in \mathbf{NP}$, by definition there exists a verifier M for L with running time $|x|^c$ for any x . Fix x and define $f(x) = (M', x)$, where M' iterates over all binary strings y of length at most $|x|^c$ and checks whether $M(x, y) = 1$ for each one of them, if $M(x, y) = 1$ for any y , then M returns '1', otherwise M returns '0'.

Clearly the language L^* cannot be computed in polynomial time, and cannot be verified in polynomial time either, that is $L^* \notin \mathbf{NP}$, so its not really interesting for our setting. This leads us to the following definition.

Definition 4.3 (NP-complete).

A language $L \subseteq \{0, 1\}^*$ is said to be NP-complete if $L \in \mathbf{NP}$ and L is NP-hard

One should be able to see now why having a polynomial algorithm for an **NPC** problem will result in $\mathbf{P} = \mathbf{NP}$, and the first step in solving this is to find such a language.

Remark 4.1.

The problem of is $\mathbf{P} = \mathbf{NPC}$ or $\mathbf{P} \neq \mathbf{NPC}$ remains an open question to this day. Whoever proves either one will be awarded 1 million dollars.

Questions 4.2.

Is there any language $L \in \mathbf{NPC}$?

Let $x_1, \dots, x_n$ be boolean variables (x_i can be assigned either 0 or 1). A boolean formula φ is said to be in conjunctive normal form (CNF) if it has the form

$$\varphi = \underbrace{(x_1 \vee x_{17} \vee \overline{x_{25}} \vee x_{80})}_{\text{clause}} \wedge \underbrace{x_9}_{\text{also clause}} \wedge \dots$$

Or between literals Literal
And between clauses

The appearances of the variable x_i are called *literals*. Each literal can be positive(x_i) or negative($\overline{x_i}$). A clause is a disjunction(OR) of literals, and the formula φ is a conjunction(AND) of these clauses. An assignment to the variables of φ evaluates to either **true** or **false**, and φ is said to be satisfiable if there is some assignment such that φ evaluates to **true**, such assignment is called *satisfying assignment*.

Definition 4.4.

$\text{CNF-SAT} := \{\varphi: \varphi \text{ is a satisfiable CNF formula}\}$

Example.

1. $\varphi := (x_1 \vee x_2) \wedge \overline{x_1} \wedge (\overline{x_2} \vee x_1 \vee x_3) \in \text{CNF-SAT}$.

$$\varphi(x_1 = F, x_2 = T, x_3 = T) = (F \vee T) \wedge T \wedge (F \vee F \vee T) = T$$

2. $\varphi := (x_1 \vee x_2) \wedge \overline{x_1} \wedge (\overline{x_2} \vee x_1 \vee x_3) \wedge \overline{x_3} \notin \text{CNF-SAT}$,
if φ_2 is satisfiable, then x_1, x_3 must have the value **F**. So that

$$\varphi_2(x_1 = F, x_3 = F) = (F \vee x_2) \wedge T \wedge (\overline{x_2} \vee F \vee F) \wedge T$$

to satisfy the first clause of φ_2 , we would need $x_2 = T$, but to satisfy the third clause, we would need $x_2 = F$, which is impossible, since x_2 cannot have value **T** and **F** at the same time. Hence, φ_2 is unsatisfiable.

Theorem 4.1 (Cook-Levin).

CNF-SAT is *npc*.

Fortunately for the students, the proof is beyond the scope of this course and will be omitted, although curious students can look at up in Computational Complexity by Aroara and Barak.

Following the discovery of the first **NPC** language, many other problems have been shown to be in **NPC** as well. The first in which we are interested is a variation of the classical CNF. For an integer $k \in \mathbb{N}$, define

$$k\text{-CNF-SAT} := \{\varphi \mid \varphi \text{ is a CNF formula in which each clause has exactly } k \text{ literals}\}.$$

Example.

$(x_1 \vee \overline{x_1}) \wedge (x_2 \vee x_3)$ is in 2-CNF-SAT,
 $(x_1 \vee x_4 \vee x_5) \wedge (x_1 \vee \overline{x_2} \vee x_3)$ is in 3-CNF-SAT.

Despite their similar definitions, there is a fundamental gap between these two problems as can be seen in the following claim:

Theorem 4.2 (proof is delegated to the practice session).
 2-CNF-SAT is in **P**.

Theorem 4.3.
 3-CNF-SAT is in **NPC**.

We split the proof into two parts: Claim 4.1 shows that 3-CNF-SAT $\in$ **NP**, and Claim 4.3 will show that 3-CNF-SAT is NP-hard.

Claim 4.1.
 3-CNF-SAT $\in$ **NP**

Proof. In order to prove that **3-CNF-SAT** $\in$ **NP** we need to provide a verifier M such that for some formula φ , and assignment a , M checks whether a satisfies φ . We provide one of many such verifier:

Algorithm 3: Verifying algorithm for 3-CNF-SAT

```

1: procedure  $M(\varphi, \alpha)$ 
2:    $\triangleright \alpha$  is the assignment i.e  $\alpha : \{x_1, \dots, x_n\} \rightarrow \{\textcolor{teal}{T}, \textcolor{red}{F}\}$ 
3:    $\triangleright$  Check if the assignment complete, i.e. every variable has an assignment
4:   for  $i \in [n]$  do
5:     if  $\alpha(x_i) = \text{undefined}$  then
6:       end
7:   end
8: end
9:    $\triangleright$  Check if any clause is unsatisfied
10:  for clause  $C \in \varphi$  do
11:    if  $C(\alpha) = \textcolor{red}{F}$  then
12:      return false
13:    end
14:  end
15:   $\triangleright$  All clauses are satisfied, hence  $\varphi(\alpha)$  is also satisfied
16:  return true

```

□

Next, we need to show that for every language $L \in \mathbf{NP}$, $L \leq_p \mathbf{3-CNF-SAT}$, which can be quite hard for us to do.

Instead we will use the fact that reductions are transitive:

Observation 4.1.

If $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$, then $L_1 \leq_p L_3$.

By using this property, we can skip the long proof, and instead we show a reduction from a known **NP**C language. The above observation follows directly from the fact that if f and g are polynomials, then $f(g(x))$ is also a polynomial.

The following figure provides a proof by picture. A formal proof is left as an exercise to the reader.

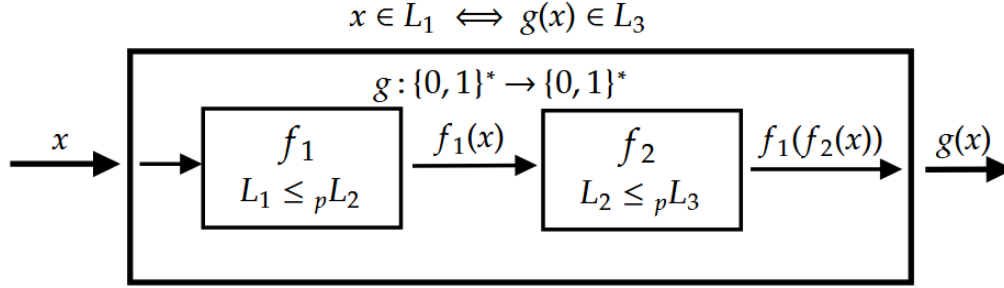


Figure 2: f_1 and f_2 are the reduction functions from $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$ respectively.

By using this property we can use the following claim

Claim 4.2.

Let $L \in \text{NP-Hard}$, and let L' be a language. If $L \leq_p L'$, then L' is also NP-Hard.

Returning to our proof:

Claim 4.3.

3-CNF-SAT is NP-hard

Proof. Using Claim 4.2 it is enough to show $\text{CNF-SAT} \leq_p \text{3-CNF-SAT}$. In order to show this, we need to define a function f such that:

1. **Running Time:** f runs in polynomial time.
2. **Correctness:** $\varphi \in \text{CNF-SAT} \Leftrightarrow f(\varphi) \in \text{3-CNF-SAT}$ for all CNF formula φ .⁵

Let $\varphi := C_1 \wedge C_2 \wedge \dots \wedge C_m$ where $C_1, \dots, C_m$ are the clauses of φ . We first define a function g that operates on a single clause of φ at a time. Then, we define f as follows.

$$f(\varphi) = \bigwedge_{i=1}^{\{m\}} g(C_i)$$

We will define g as follows: for each clause $C := l_1 \vee l_2 \dots \vee l_k$ of φ , we will replace it by a *gadget* of clauses according to the following rules:

1. If $k = 3$, then return C as it. i.e.

$$g(l_1 \vee l_2 \vee l_3) = l_1 \vee l_2 \vee l_3$$

⁵We can always assume valid input, in our case that means that φ is in CNF form, but we cannot assume that it is satisfiable.

2. If $k < 3$, then repeat one of the literals until the clause has exactly 3 literals. For example

$$g(l_1 \vee l_2) = l_1 \vee l_2 \vee l_2.$$

3. If $k > 3$ then create $k - 3$ **new** variables named $y_1, \dots, y_{k-3}$ and let:

$$g(l_1 \vee l_2 \vee \dots \vee l_k) = (l_1 \vee l_2 \vee y_1) \wedge (\overline{y_1} \vee l_3 \vee y_2) \wedge (\overline{y_2} \vee l_4 \vee y_3) \wedge \dots \wedge (\overline{y_{k-3}} \vee l_{k-1} \vee l_k).$$

By definition all cases return a valid CNF-form formula. It is also easy to see that the first two cases take a constant amount of time. The third case requires as to create $k - 3$ new variables and $k - 2$ new clauses each of length 3, so that the third step takes $O(n)$ time. As f is g running m times, the running time of f is still polynomial.

It remains to show the correctness of f , i.e.

$$\varphi \in \text{CNF-SAT} \Leftrightarrow f(\varphi) \in 3 - \text{CNF-SAT}$$

We assume that φ is a CNF formula with variable $x_1, \dots, x_n$ and clauses $C_1, \dots, C_m$.

$\Rightarrow$ (Completeness): Assume that $\varphi \in \text{CNF-SAT}$, This means there exists a satisfying assignment α for φ . Our goal is to show that a satisfying assignment α' exists for $f(\varphi)$, we prove this by constructing α' . Our assignment will copy the original assignment for the original variable of φ that is

$$\forall i \in [n] : \alpha'(x_i) = \alpha(x_i).$$

The **new variables** we of $f(\varphi)$ will be dealt later.

Fix a clause $C \in \varphi$ we show that $g(C)$ can be satisfied as well. If C we have $|C| \leq 3$, then we are done as $C \equiv g(C)$, and by assumption $C[\alpha] = \textcolor{teal}{T}$ so that we have $g(C)[\alpha'] = \textcolor{teal}{T}$.

Example.

1. $C = x_1 \vee x_3 \vee \overline{x_{17}}; \quad \alpha(x_1) = \textcolor{teal}{T}, \alpha(x_3) = \textcolor{red}{F}, \alpha(x_{17}) = \textcolor{teal}{T}$
 $\Rightarrow C[\alpha] = \textcolor{teal}{T} \vee \textcolor{red}{F} \vee \textcolor{red}{F}$
 $\Rightarrow g(C)[\alpha'] = \textcolor{teal}{T} \vee \textcolor{red}{F} \vee \textcolor{red}{F}$
2. $C = x_1 \vee \overline{x_{17}}; \quad \alpha(x_1) = \textcolor{red}{F}, \alpha(x_{17}) = \textcolor{red}{F}$
 $\Rightarrow C[\alpha] = \textcolor{red}{F} \vee \textcolor{teal}{T}$
 $\Rightarrow g(C)(\alpha') = (x_1 \vee \overline{x_{17}} \vee \overline{x_{17}})(\alpha') = \textcolor{red}{F} \vee \textcolor{teal}{T} \vee \textcolor{teal}{T}$

Otherwise let $C = l_1 \vee l_2 \vee \dots \vee l_k$ where $k \geq 4$ be such clause. Then, by assumption $C[\alpha] = \textcolor{teal}{T}$, so that there exists some $i \in [k]$ such that $\ell_i = \textcolor{teal}{T}$. Since for all $j \in [k - 3]$ the value of $\alpha'(y_j)$ have not yet been set, we define them as follows:

$$\alpha'(y_j) = \begin{cases} \textcolor{teal}{T} & \text{if } j \leq i - 2, \\ \textcolor{red}{F} & \text{if } j > i - 2. \end{cases}$$

Then,

$$\begin{aligned} g(C)[\alpha'] &= (l_1 \vee l_2 \vee y_1) \wedge \dots \wedge (\overline{y_{i-2}} \vee l_i \vee y_{i-1}) \wedge \dots \wedge (\overline{y_{k-3}} \vee l_{k-1} \vee l_k)[\alpha'] \\ &= (? \vee ? \vee \textcolor{teal}{T}) \wedge \dots \wedge (\textcolor{red}{F} \vee \textcolor{teal}{T} \vee \textcolor{teal}{T}) \wedge \dots \wedge (\textcolor{teal}{T} \vee ? \vee ?) = \textcolor{teal}{T}. \end{aligned}$$

as required.

$\Leftarrow$ (Soundness): Assume that $f(\varphi) \in 3\text{-CNF-SAT}$, This implies there exists a satisfying assignment α' for $f(\varphi)$, We must show that there exists a satisfying assignment α for φ . We argue that copying the assignment of the original variables from α' to α will produce a satisfying assignment for φ . To see this, assume toward a contradiction that α is not a satisfying assignment for φ . Then, there must exist a clause $C = l_1 \vee l_2 \dots \vee l_k$ such that $C[\alpha] = F$. If $k \leq 3$, then since $g(C) \equiv C$, it follows that $T = g(C)[\alpha'] = C[\alpha'] = C[\alpha] = F$ which is a contradiction to the assumption that α satisfies $f(\varphi)$. Otherwise assume that $k \geq 4$, by assumption

$$C[\alpha] = l_1 \vee l_2 \vee \dots \vee l_k = F$$

meaning that for all $i \in [k]$ we have $l_i = F$. On the otherhand, since $f(\varphi)[\alpha']$ is satisfied the gadget clause

$$g(C) = (l_1 \vee l_2 \vee y_1) \wedge (\overline{y_1} \vee l_3 \vee y_2) \wedge (\overline{y_2} \vee l_4 \vee y_3) \wedge \dots \wedge (\overline{y_{k-3}} \vee l_{k-1} \vee l_k)$$

is also satisfied, and such every clause should be T . In order for the first clause to be T , $y_1 = T$ must hold. In order to satisfy the next clause $y_2 = T$ must hold, following this argument one can see that $y_i = T$ must hold for all $i \in [k-3]$. Looking at the last clause, $l_{k-1} = l_k = F$ by assumption, and $\overline{y_{k-3}} = F$ in order for any other clause to be satisfied, which is a contradiction to the assumption that α' satisfies φ .⁶

□

4.1. Independent set

For a graph G , let $\alpha(G)$ denote the size of the maximum independent set in G . Define:

Definition 4.1.1.

$\text{IS} := \{ \langle G, k \rangle : \alpha(G) \geq k \}$.

Theorem 4.1.1.

IS is in **NPC**.

Proof. We show that $3\text{-CNF-SAT} \leq_p \text{IS}$, proving that $\text{IS} \in \mathbf{NP}$ is left as homework.⁷ Here the reduction might seem a little confusing at first, we are translating a formula into a graph and a number. Given a 3-CNF formula φ , we construct a graph G_φ as follows:

1. **Triangles:** For each clause l_1, l_2, l_3 we create a triangle with 3 vertices named $v_{l_1}, v_{l_2}, v_{l_3}$.
2. **Consistency Edges:** For any pair of complementary literals $x_j, \overline{x_j}$ that are in different clauses, put an edge between the vertices that correspond to the literals.

⁶Note that our argument shows that for any assignment of $y_1, \dots, y_{k-3}$ the value of $g(C)(\alpha') = F$.

⁷The Verifying algorithm of IS is identical to Clique, but we check that the edges do not exist.

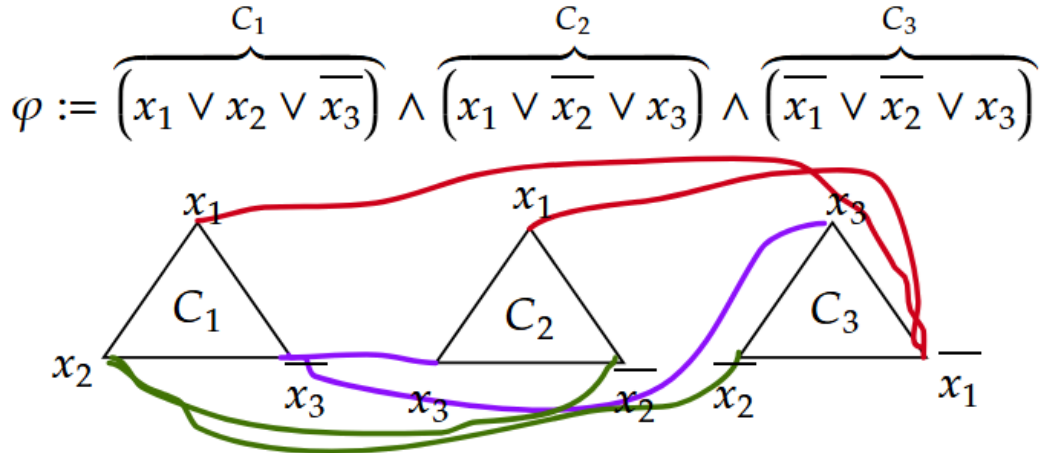


Figure 3: Here $f(\varphi) = \langle G_\varphi, m \rangle$ where G_φ is the graph above with 9 vertices and $m = 3$ is the number of clauses in φ .

We return the pair $\langle G_\varphi, m \rangle$ where m is the number of clauses. While not intuitive at first, the number m is chosen because an independent set can contain at most one vertex from each triangle.

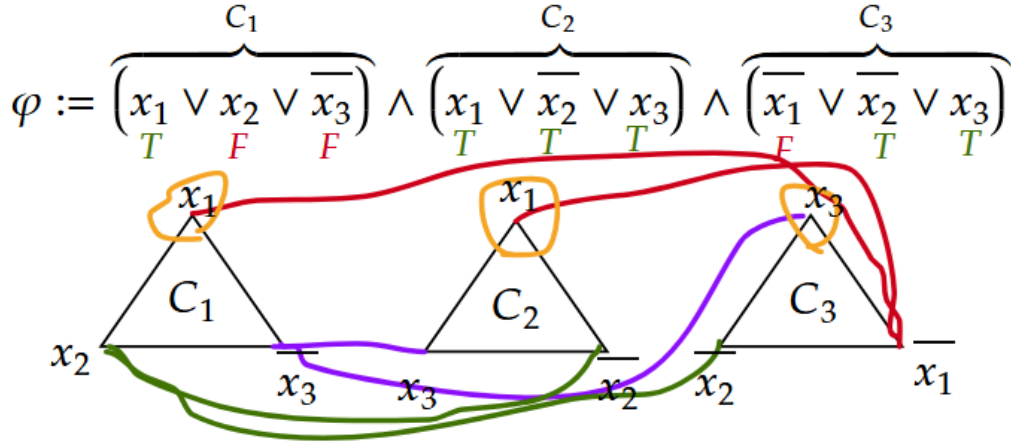


Figure 4: Taking the vertices that correspond to a T literal in each triangle results in an independent set of size m .

This algorithm indeed runs in polynomial time, as looping through all the clauses is linear in the number of clauses, and the maximum number of edges one can add is n^2 . Now we need to prove

$$\varphi \in 3\text{-CNF-SAT} \iff \langle G_\varphi, m \rangle \in \text{IS}$$

$\implies$: Let φ be satisfiable and let α_φ be a satisfying assignment for φ . As α_φ satisfies φ , at least one literal of each clause is satisfied, pick any one such literal from each clause. The set of vertices corresponding to the set of literals chosen is independent in G_φ and has size of m .

(See Figure 4 as an example.)

$\impliedby$: Suppose G_φ has an independent set of size m , and let S be such an independent set. Let $l_1, \dots, l_m$ denote the vertices of S , for each $i \in [m]$ let $v(l_i) \in \{x_1, \overline{x_1}, \dots, x_n, \overline{x_n}\}$ be the variable corresponding to l_i . Set $v(l_i) = T$ for all $i \in [m]$. Finally if $x_i \in [n]$ did not receive an assignment set $x_i = T$.

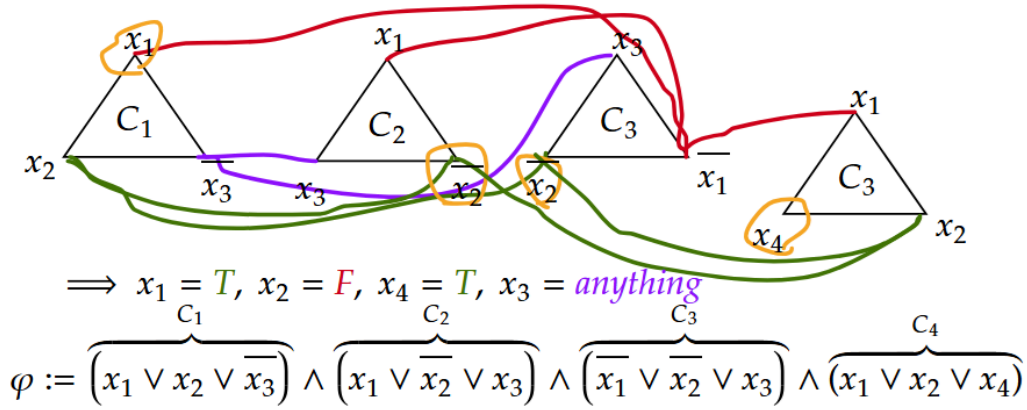


Figure 5: Here the orange vertices yield an independent set of size 4, and we set the value of each vertex to be T , which gives us a partial assignment that is already satisfying.

It remains to show that this assignment is both **satisfying** and **consistent**.

First, the assignment is satisfying. Each vertex l_i correspond to a single literal in a unique clause of φ . By construct $l_i = T$, and therefore every clause contains at least one true literal and is satisfied.

By **consistent** we mean that each variable $x_i \in [n]$ recieved **exactly one assignment**. In principle, it could happen that there exist $l_i, l_j \in S$ with $i \neq j$ such that $v(l_i) = v(l_j)$ (assume w.l.o.g. that $v(l_i) = x_k$ and $v(l_j) = \overline{x_k}$). In that case, the construction would force us to set both $x_k = T$ and $\overline{x_k} = T \Rightarrow x_k = F$ which is **inconsistent**. However, this situation cannot occur, if $v(l_i) = v(l_j)$, then the vertices l_i and l_j are adjacent in G_φ , i.e. $(l_i, l_j) \in E(G_\varphi)$. This contradicts the assumption that S is an independent set. Therefore, each variable recieve a consistent assignment, and we conclude that φ is satisfiable. $\square$

4.2. Graph coloring

For a graph G denote by $\chi(G)$ the least $k \in \mathbb{N}$ such that G is k -colorable.

Definition 4.2.1.

$$k\text{-COL} := \{G : \chi(G) \leq k\}$$

It is well known that $2\text{-COL} \in \mathbf{P}$.

Theorem 4.2.1.

$3\text{-COL} \in \mathbf{NPC}$.

In order to prove the theorem, we introduce a new **NPC** language. Let φ be a formula, φ is said to be *not all equal satisfiable* (NAE-SAT) if it has a satisfying assignment such that in each clause it has at least one satisfied literal and at least one that is not satisfied.

Definition 4.2.2 (NAE- k -CNF-SAT).

$\text{NAE-}k\text{-CNF-SAT} := \{\varphi : \varphi \text{ is NAE-SAT, with exactly } k \text{ literals in each clause}\}.$

The proof that NAE- k -CNF-SAT is NP-Complete is left to the practice sessions. We are ready to start our proof.

Proof. We skip again the proof that $3\text{-COL} \in \mathbf{NP}$, which is left as homework, and show that $k\text{-NAE-SAT} \leq_p 3\text{-COL}$. Given a 3-CNF formula φ , define G_φ as follows:

1. Start with a single vertex D . This is our *Don't care vertex*.
2. For each variable x_i of φ , add two new vertices $x_i, \bar{x}_i$, add an edge between them, and connect both to D . This are our *variable gadgets*.
3. For each clause $l_1 \vee l_2 \vee l_3$ we create a triangle with 3 vertices named l_1, l_2, l_3 , this is our *clause gadget*.
4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.

We skip the proof that the algorithm runs in polynomial time. It remains to prove that

$$\varphi \in \text{NAE-}k\text{-CNF-SAT} \Leftrightarrow G_\varphi \in 3\text{-COL}$$

$\Rightarrow$: Given a satisfying NAE assignment α_φ for φ , define the following 3-coloring of G_φ :

- D will be colored as D
- For each variable x_i , if x_i is assigned **true** under α_φ color x_i as **T** and $\bar{x}_i$ in **F**, otherwise color x_i as **F** and $\bar{x}_i$ in **T**
- For each clause gadget, scan the corresponding clause c , color first literal that is assigned **true** with **T**, the first that assigned **false** with **F**, and color the vertex that was left with D .

It is clear that edges inside vertex/clause gadgets have both ends in different colors, it remains to show that edges between vertex and clause gadgets has its ends colored in different colors. without loss of generality, let x be a variable assigned **true** by α_φ . As α_φ is proper, all of the vertices bearing $\bar{x}$ found in a clause gadgets are colored either **F** or D , and the claim follows.

$\Leftarrow$: Given a 3-coloring ψ of G_φ , we define a NAE-satisfying assignment for φ . As all of the variable gadgets form a triangles with a common vertex D , it leaves them with two colors to be chosen. Take the variable gadget for an arbitrary variable x and set the color under x to be **true** and the color under $\bar{x}$ to be **false**. This defines a valid assignment to the variables of φ . The assignment is NAE as each clause gadget has one variable colored **true** and one **false**. $\square$

4.3. Max-cut

Given a graph G , a *cut* is defined as the set of edges between $S \subseteq V(G)$ and $\bar{S} = V(G) \setminus S$. We denote the set of edges by $E_G(S, \bar{S}) := \{(u, v) : (u, v) \in E(G), u \in S, v \in \bar{S}\}$, and the number of edges by $e_G(S, \bar{S}) := |E_G(S, \bar{S})|$. Denote by $\sigma(G) := \max_{S \subseteq V(G)} e_G(S, \bar{S})$.

Definition 4.3.1 (MAX-CUT).

$\text{MAX-CUT} := \{ \langle G, k \rangle : \sigma(G) \geq k \}$

Theorem 4.3.1.

$\text{MAX-CUT} \in \mathbf{NPC}$

Proof. We again skip the proof that $\text{MAX-CUT} \in \mathbf{NP}$, and show that $\text{MAX-CUT} \leq_p 3\text{-COL}$. Given a 3-CNF formula φ , define G_φ as follows:

1. For each variable x_i of φ , add two new vertices $x_i, \bar{x}_i$ and an edge between them. These are our *variable gadgets*.
2. For each clause $l_1 \vee l_2 \vee l_3$, we create a triangle with vertices l_1, l_2, l_3 ; this is our *clause gadget*.
3. For each literal vertex in a clause gadget, connect it to the complementary literal vertex in the variable gadget.

(This is the same as the NAE- k -CNF-SAT reduction to 3-COL but without the vertex D). We return the pair $\langle G_\varphi, n + 5m \rangle$. We skip the proof that the algorithm runs in polynomial time. It remains to prove that

$$\varphi \in \text{NAE-}k\text{-CNF-SAT} \Leftrightarrow \langle G_\varphi, n + 5m \rangle \in \text{MAX-CUT}$$

$\Rightarrow$: Given a satisfying NAE assignment α_φ for φ , define $S \subseteq V(G_\varphi)$ to consist of all vertices whose label is a literal assigned **true** under α_φ . As α_φ is consistent, all variable gadgets must cross $(S, \bar{S})$ adding n edges to the cut. As α_φ is a valid NAE assignment, at least two edges cross $(S, \bar{S})$ in every clause gadget, adding $2m$ edges to the cut. Each edge between a variable and clause gadget has the form $(l, \bar{l})$, which means it is also in the cut, adding $3m$ edges to the cut. Overall, we count at least $n + 5m$ edges.

$\Leftarrow$: Suppose that $\langle G_\varphi, n + 5m \rangle \in \text{MAX-CUT}$. Let $(S, \bar{S})$ be a cut of G_φ such that $e_{G_\varphi}(S, \bar{S}) = n + 5m$. Define the assignment α_φ for φ in which all variable gadget literals found in S are assigned **true** and all remaining are assigned **false**. This defines a consistent assignment. It remains to prove that α_φ is NAE-satisfying. Every edge between a variable and clause gadget is also in the cut and has the form $(l, \bar{l})$. Because $e_{G_\varphi}(S, \bar{S}) = n + 5m$, each clause gadget has 2 edges crossing $(S, \bar{S})$, meaning each gadget has at least one vertex in S and one in $\bar{S}$. Take a vertex l in a clause gadget that is in S . Then the edge $(l, \bar{l})$ implies that $\bar{l} \in \bar{S}$, meaning the literal corresponding to l is assigned **true**. In a similar manner, if l is in $\bar{S}$ then the edge $(l, \bar{l})$ means the $\bar{l} \in S$ implying that l is assigned **false**. $\square$